Текст отчета.

В рамках выполнения задания было разработано небольшое аналитическое приложение на языке программирования Python с использованием библиотеки Plotly Dash. Основной целью работы являлось создание веб-интерфейса, позволяющего визуализировать данные и демонстрировать возможности построения аналитических дашбордов.

В ходе выполнения работы было реализовано приложение, которое загружает данные из файлов формата CSV, выполняет их обработку и отображает результаты в виде графиков и таблиц. Для обработки данных использовалась библиотека Pandas, которая обеспечивает удобную загрузку и анализ табличной информации.

Интерфейс приложения разработан с использованием библиотеки Dash. Данная библиотека позволяет создавать веб-приложения на Python и отображать интерактивные графики, построенные при помощи библиотеки Plotly. В интерфейсе реализовано несколько элементов визуализации данных, включая круговую диаграмму распределения данных, столбчатые диаграммы и гистограмму.

Данные для анализа хранятся в CSV файлах внутри проекта и загружаются при запуске приложения. После загрузки данные обрабатываются и используются для построения графиков и таблиц. В интерфейсе реализован элемент фильтрации, позволяющий выбирать отображаемые данные.

Структура проекта организована по стандартному принципу для приложений Dash. Основной файл приложения содержит код запуска сервера и построения интерфейса. Данные находятся в отдельной папке проекта. Стили интерфейса подключаются через папку assets. В проекте также присутствует файл requirements.txt со списком используемых библиотек.

Для проверки корректности работы приложения добавлен тест загрузки интерфейса. Также реализован сценарий нагрузочного тестирования, позволяющий выполнить базовую проверку работы приложения.

Разработка и запуск приложения выполнялись на локальном компьютере с использованием Python. После установки зависимостей приложение запускается через основной файл и становится доступным в браузере по локальному адресу.

Таким образом было создано аналитическое веб-приложение, демонстрирующее возможности построения дашбордов и визуализации данных с использованием Plotly Dash.

Общая структура репозитория

Проект состоит из нескольких логических частей:

1. основной файл приложения
2. файлы зависимостей
3. стили интерфейса
4. данные для анализа
5. тестовые файлы

Основной файл приложения app.py

Файл app.py является главным исполняемым файлом проекта. В нем реализована основная логика работы приложения.

В данном файле выполняются следующие задачи:

- подключение используемых библиотек Python
- загрузка данных из файлов формата CSV
- обработка и подготовка данных для визуализации
- построение графиков и диаграмм при помощи библиотеки Plotly
- формирование пользовательского интерфейса на основе библиотеки Dash
- реализация элементов взаимодействия пользователя с интерфейсом
- запуск локального веб-сервера приложения

После запуска файла app.py приложение становится доступным через браузер по локальному адресу. Пользователь может просматривать графики, таблицы и выполнять фильтрацию данных.

Таким образом, файл app.py объединяет все основные элементы системы и обеспечивает функционирование аналитической панели.

Файл зависимостей requirements.txt

Файл requirements.txt содержит список библиотек Python, которые используются в проекте.

Пример содержимого данного файла:

```
dash
pandas
plotly
pytest
playwright
locust
```

Данный файл необходим для того, чтобы обеспечить корректную установку всех зависимостей проекта на любом компьютере. Установка библиотек выполняется одной командой:

pip install -r requirements.txt

Использование данного файла делает проект воспроизводимым и облегчает его запуск в другой среде разработки.

Каталог assets

Каталог assets используется для хранения файлов оформления интерфейса приложения.

Внутри данного каталога расположен файл:

style.css

Библиотека Dash автоматически подключает файлы из данной папки и применяет их для оформления интерфейса.

Файл оформления style.css

Файл style.css содержит правила визуального оформления интерфейса приложения.

В данном файле задаются параметры отображения элементов страницы, в том числе:

- стиль шрифтов
- цвет фона страницы
- расположение графиков
- отступы между элементами
- оформление карточек интерфейса
- выравнивание заголовков

Использование отдельного файла стилей позволяет отделить программную логику приложения от визуального оформления интерфейса.

Каталог data

Каталог data предназначен для хранения исходных данных, используемых приложением.

Все данные представлены в формате CSV и загружаются в приложение при его запуске.

В каталоге расположены следующие файлы:

characters.csv
actions.csv
dice_rolls.csv
encounters.csv

Каждый из этих файлов содержит определённый набор данных, который используется для анализа и визуализации.

Файл characters.csv

Файл characters.csv содержит данные о персонажах. В нем представлены основные характеристики сущностей, участвующих в системе.

Типичная структура файла включает следующие поля:

- идентификатор персонажа
- имя персонажа
- класс персонажа
- показатель здоровья
- значение атаки
- значение защиты

Данные из этого файла используются для построения статистики по характеристикам персонажей и распределению их классов.

Файл actions.csv

Файл actions.csv содержит сведения о действиях, выполняемых персонажами.

В таблице представлены следующие данные:

- идентификатор действия
- персонаж, выполняющий действие
- тип действия
- величина нанесённого урона
- цель действия

Данные из данного файла используются для анализа активности персонажей и расчёта статистики по выполняемым действиям.

Файл dice_rolls.csv

Файл dice_rolls.csv содержит результаты бросков кубов.

В таблице представлены следующие параметры:

- идентификатор записи
- персонаж
- тип куба
- результат броска

Данные используются для построения гистограммы распределения результатов бросков и демонстрации вероятностного характера игровых событий.

Файл encounters.csv

Файл encounters.csv содержит информацию о игровых событиях или столкновениях.

В нем представлены следующие данные:

- идентификатор события
- дата проведения
- ведущий игры
- место проведения события
- результат столкновения

Данные из данного файла отображаются в виде таблицы и могут фильтроваться пользователем через элементы интерфейса.

Каталог tests

Каталог tests содержит файлы тестирования приложения.

Наличие тестов в проекте позволяет проверить корректность работы интерфейса и базовую устойчивость системы.

В данном каталоге находятся два файла:

test_ui.py
locustfile.py

Файл test_ui.py

Файл test_ui.py содержит автоматизированный тест пользовательского интерфейса.

В данном тесте используется библиотека Playwright. Тест выполняет следующие действия:

- автоматически открывает браузер
- переходит на страницу приложения

- проверяет наличие ключевых элементов интерфейса

Данный тест позволяет убедиться, что веб-интерфейс приложения успешно запускается и отображается корректно.

Файл locustfile.py

Файл locustfile.py предназначен для выполнения нагрузочного тестирования.

В нем описан сценарий, при котором виртуальные пользователи выполняют обращения к веб-приложению. Такой тест позволяет оценить доступность приложения и проверить его поведение при множественных запросах.

Алгоритм рассмотрения и демонстрации.

1) Войти в Codespace.

2) Вбить:

1. «ls»
   @Dietwing ➜ /workspaces/pro.prac.1 (main) $ ls

   README.md project_practicum_task1_repository

2. «cd project_practicum_task1_repository»
   @Dietwing ➜ /workspaces/pro.prac.1 (main) $ cd project_practicum_task1_repository

3. «ls battle_analytics_dashboard»
   @Dietwing ➜ /workspaces/pro.prac.1/project_practicum_task1_repository (main) $ ls battle_analytics_dashboard

4. «cd battle_analyticks_dashboard»
   @Dietwing ➜ /workspaces/pro.prac.1/project_practicum_task1_repository (main) $ cd battle_analyticks_dashboard

   bash: cd: battle_analyticks_dashboard: No such file or directory

5. «cd battle_analytics_dashboard»
   @Dietwing ➜ /workspaces/pro.prac.1/project_practicum_task1_repository (main) $ cd battle_analytics_dashboard

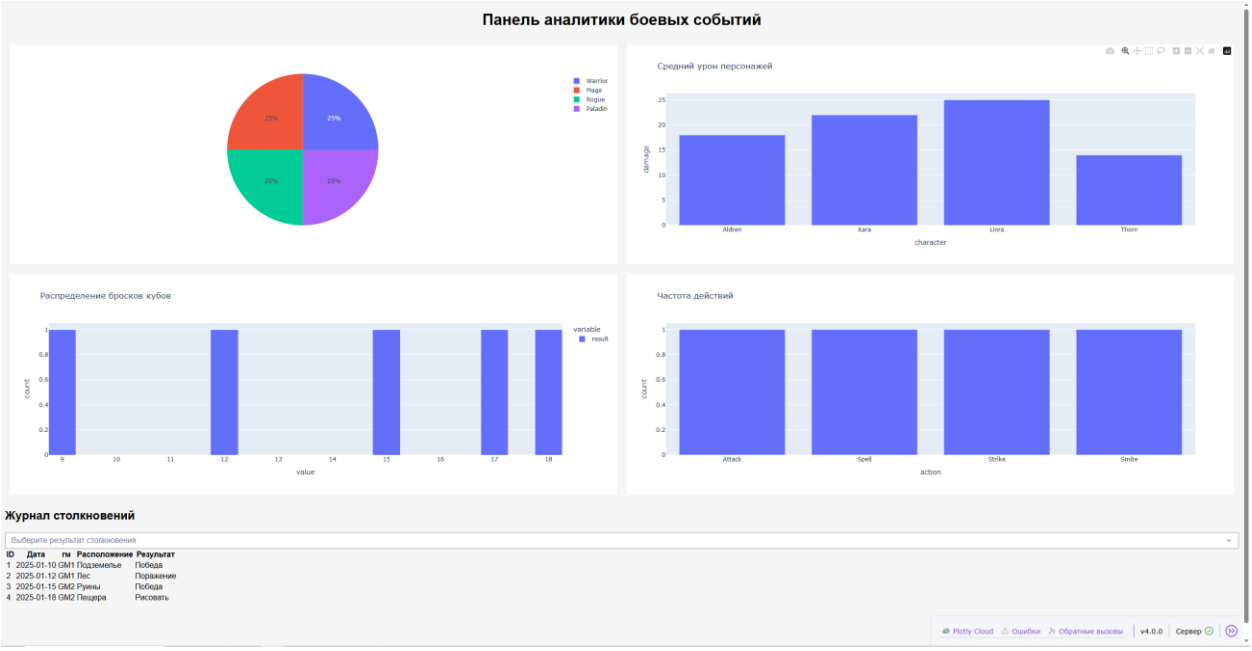
6. «ls»
   @Dietwing ➜ /workspaces/pro.prac.1/project_practicum_task1_repository/battle_analytics_dashboard (main) $ ls

   app.py assets data requirements.txt tests

7. «pip install dash pandas plotly pytest playwright locust»
   @Dietwing ➜ /workspaces/pro.prac.1/project_practicum_task1_repository/battle_analytics_dashboard (main) $ pip install dash pandas plotly pytest playwright locust

   … «Загрузка» …

8. «python app.py»
   @Dietwing ➜ /workspaces/pro.prac.1/project_practicum_task1_repository/battle_analytics_dashboard (main) $ python app.py

3) Перейти в порты и перейти по ссылки на локальный сервер.

4) Узреть результат: